知乎 关注 推荐 热榜 专栏 圈子 New 付费咨询 知学堂 AI圈养完龙虾又开始养骡子了 直答 消息

[总结] 漫谈HDR和色彩管理（五）游戏中的HDR

Lele Feng
http://candycat1992.github.io/

关注她

收录于 · RGBA ›

283 人赞同了该文章 ›

终于到了完结篇！写了这么多篇，是时候回到最开始的初心——在游戏里使用HDR显示。从2016年以来，各大游戏厂商陆续开始在自家游戏里支持HDR显示。从早期的声名狼藉系列到使用UE4开发的战争机器5，目前市场上3A游戏对HDR显示的支持基本已经是标配。最后这一节，我们先看COD分享的实现方式，再对比UE4目前对HDR的实现。

HDR in COD: WW II

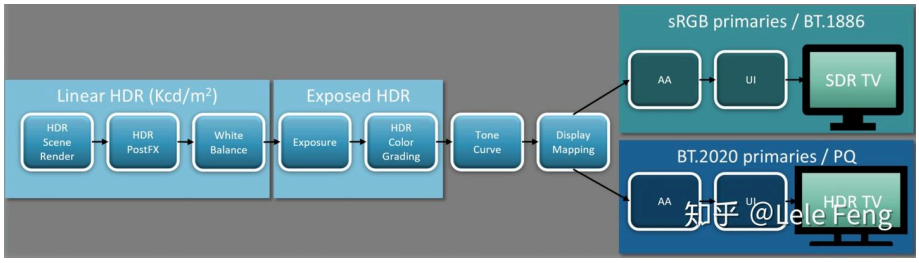
游戏中的HDR显示套用了ACES的很多概念和实现，尤其是Output Transform（RRT + ODT）部分。但ACES中许多针对影视流程的概念其实我们并不需要，比如游戏基本不需要考虑复杂的IDT，基本就是sRGB纹理的输入和输出转换就足够了。COD: WW II也是在ACES的基础上针对自己的HDR颜色管线做出了自己的调整。

在COD的技术分享中作者提到，做HDR显示的实现目标是：

- 保持HDR和SDR的视觉一致性
- 在新的SDR管线下可以得到与原先老的SDR类似的效果
- 质量高且性能高效，尽可能给美术团队少找麻烦

新的颜色管线

怀着这样的目标，COD: WW II的新颜色管线如下所示：



来源：HDR in Call of Duty

与COD先前的管线相比，新的HDR管线有几个重要的改动：

- 后处理从原先的sRGB gamma空间移到了HDR linear空间下，以保持HDR和SDR的视觉一致性
- 之后应用曝光参数，将整个场景亮度进行缩放，进入Exposed HDR空间。该空间仍然是线性空间，但同时会使用一次颜色转换矩阵将颜色空间从sRGB转换到Rec. 2020颜色空间下
- 在Rec. 2020线性颜色空间下应用Color Grading
- 应用Tonemapping。Tonemapping目前分成了两个操作，先运用一个固定的Tone Curve来进行统一的图像转换（相当于ACES中的RRT），再使用一个display mapping函数输出到特定的输出设备上（相当于ACES中的ODT）。这两步相当于ACES中的Output Transform，即RRT + ODT

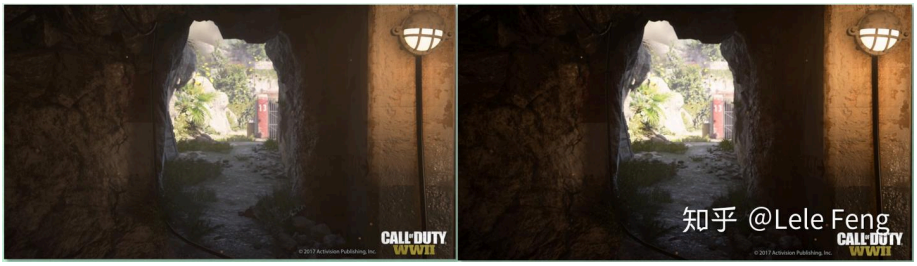
▲ 赞同 283 ▼ 41 条评论 389 47 分享 申请转载 ···

关于作者
Lele Feng
http://candycat1992.githu...
回答 8 文章 23 关注者 25,440
关注她 发私信

大家都在搜 换一换
伊朗局势 553 万 热
中南大学湘雅医院一研究... 388 万 热
小红书被曝存在儿童软色... 328 万 热
美以袭击伊朗 313 万 热
湘雅医院坠江身亡研究生... 308 万
中南大学湘雅医院 302 万
以军刺杀伊朗高官拉里贾尼 299 万 热
永辉喊话山姆不要让供应... 250 万
樊振东加盟杜塞尔多夫 245 万
省考第一名被指P高14分... 244 万

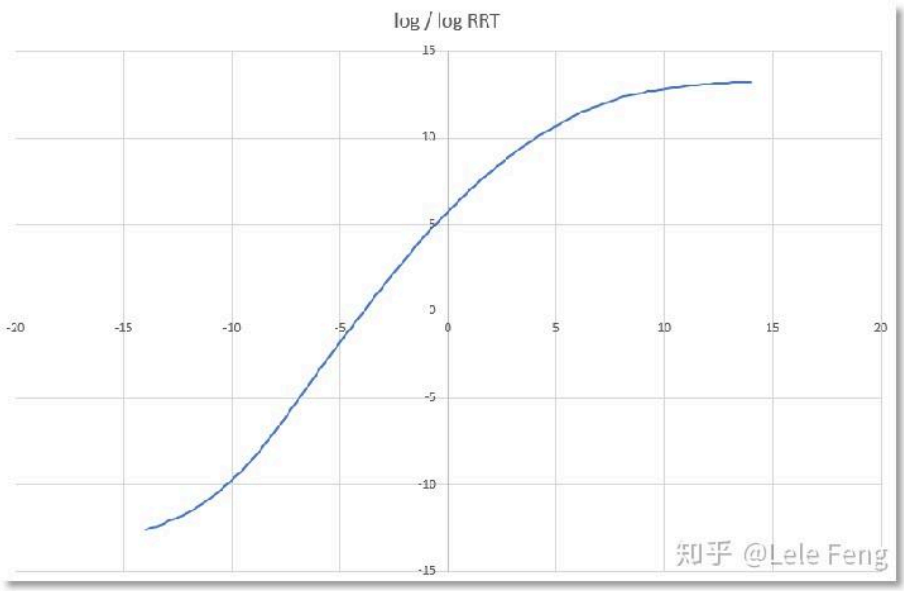
知乎 关注 推荐 热榜 专栏 圈子 New 付费咨询 知学堂 直答

相当于ACES的RRT阶段，作者使用了一个固定的Tone Curve来将HDR场景亮度重新映射到PQ对应的0~10000尼特的输出亮度范围内，它本质上是一条S曲线。这条S曲线的定义更多取决于想要的艺术表现，依靠这样一条S曲线可以得到和老的SDR下filmic tonemap类似的效果。以下是应用Tone Curve前后的画面对比，可以看到应用之后的画面更具有“电影感”：



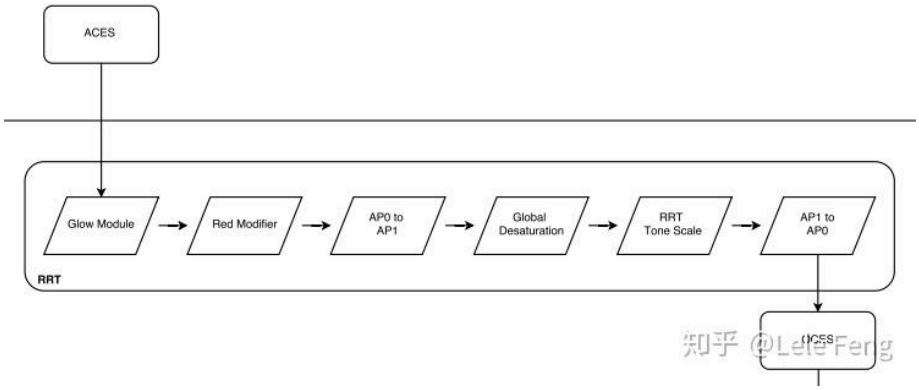
来源：HDR in Call of Duty

这条S曲线如下所示，横纵坐标分别是对亮度取log之后的stop值，它的输入颜色空间是ACEScg，即AP1空间：



来源：HDR in Call of Duty

COD最后使用的这条曲线非常接近原生ACES RRT中使用的tonescale曲线，这条RRT tonescale曲线本身是由分段的B-spine曲线定义而得（详见ACESlib.Tonescales.ctl文件中的segmented_spline_c5_fwd函数）。为了完整性这里给出ACES RRT完整的工作流如下所示：



至此，所有风格和效果上的计算都已完成，整个场景亮度被重新映射到0~10000尼特范围内。现在，我们可以在这个输出亮度范围内真正输出我们的图像了。

Display Mapping

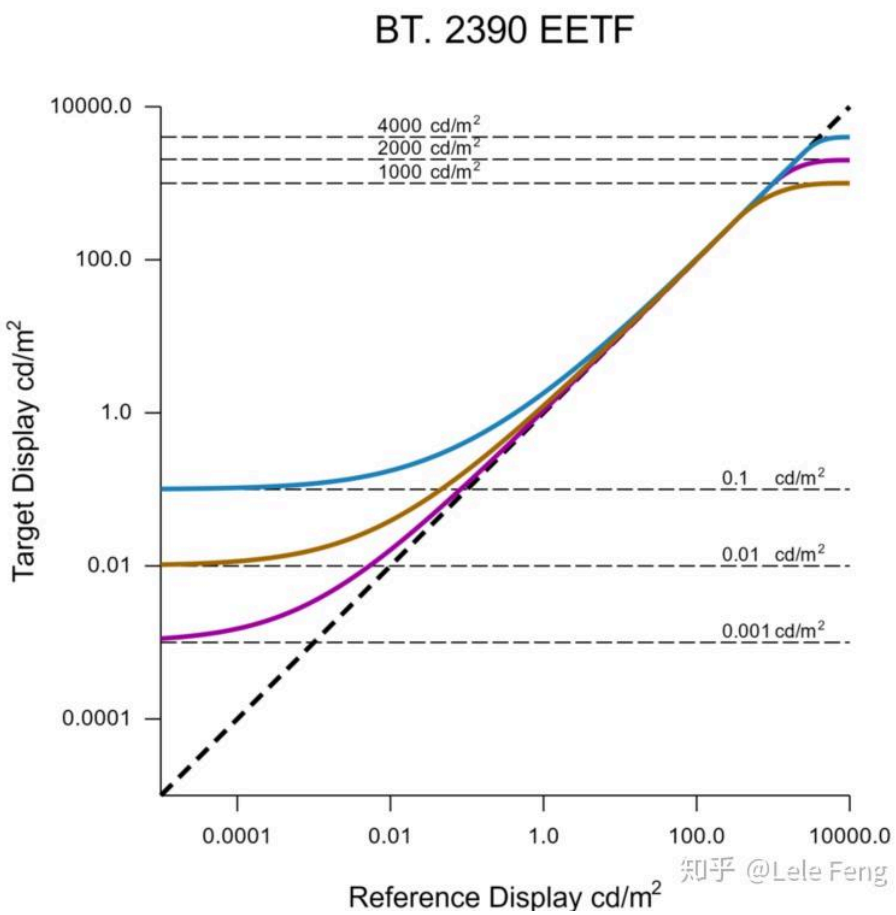
知乎 关注 推荐 热榜 专栏 圈子New 付费咨询 知学堂 直答

- HDR Range Reduction：0~10000尼特的亮度范围对白制作图上的显示设备来说范围太大了，因此需要使用以下几种操作来为输出设备降低亮度范围
- Color Space Transformation：使用颜色转换将颜色空间最终转换到输出设备的颜色空间下

Display Mapping可以分为SDR和HDR显示两条线，我们先来看HDR显示下的Display Mapping，它的两个部分分别为：

- 使用BT. 2390的EETF和用户校准的方式计算HDR Range Reduction
- 使用PQ OETF将结果最终转换到输出设备的颜色空间下

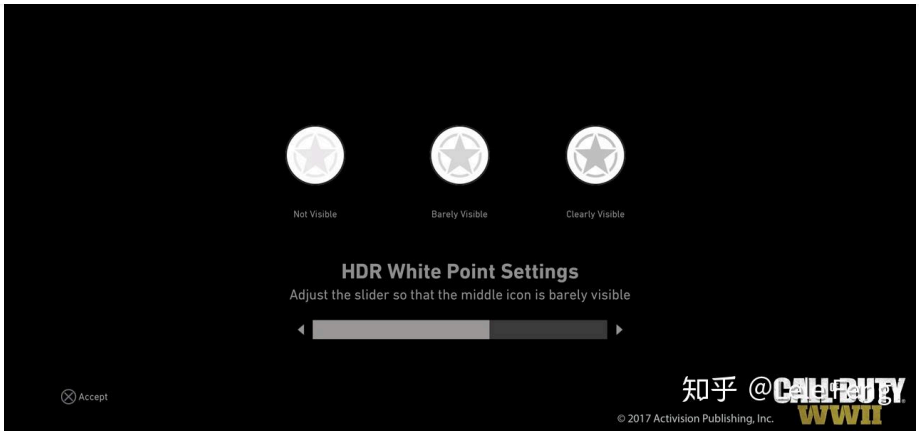
首先是HDR Range Reduction，这一步的目的是把0~10000尼特映射到用户HDR显示设备真正的黑点和白点范围内。作者选择使用BT. 2390的EETF传递函数来完成这个映射。BT. 2390的EETF可以根据指定的最小和最大亮度值，通过对ICtCp空间下的亮度值应用一条Hermite曲线来调整输出的亮度范围。以下是给定不同的最大最小亮度值时的函数曲线：



来源：HDR in Call of Duty

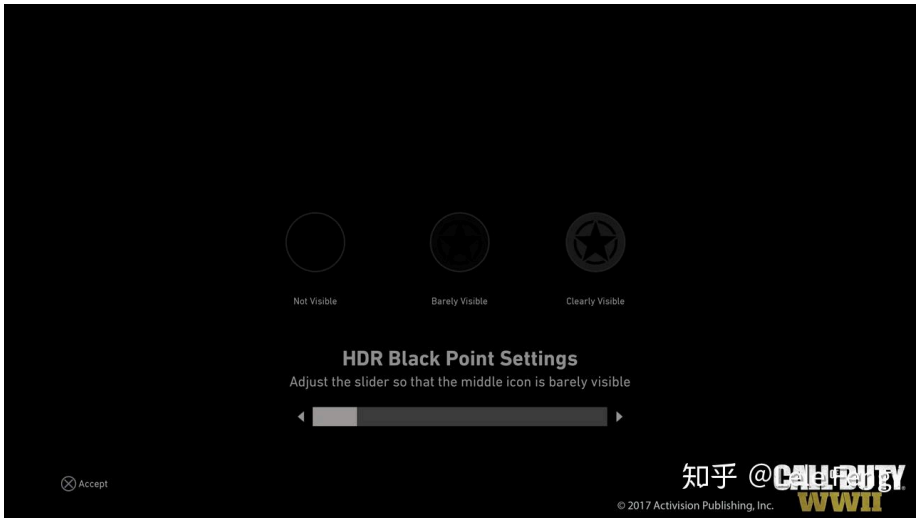
BT2390的EETF的输入——最小和最大亮度值，则是通过显示校准功能让用户进行设置的。HDR白点设置界面如下：

知乎 关注 推荐 热榜 专栏 圈子 New 付费咨询 知学堂 直答



来源：HDR in Call of Duty

HDR黑点设置如下：



来源：HDR in Call of Duty

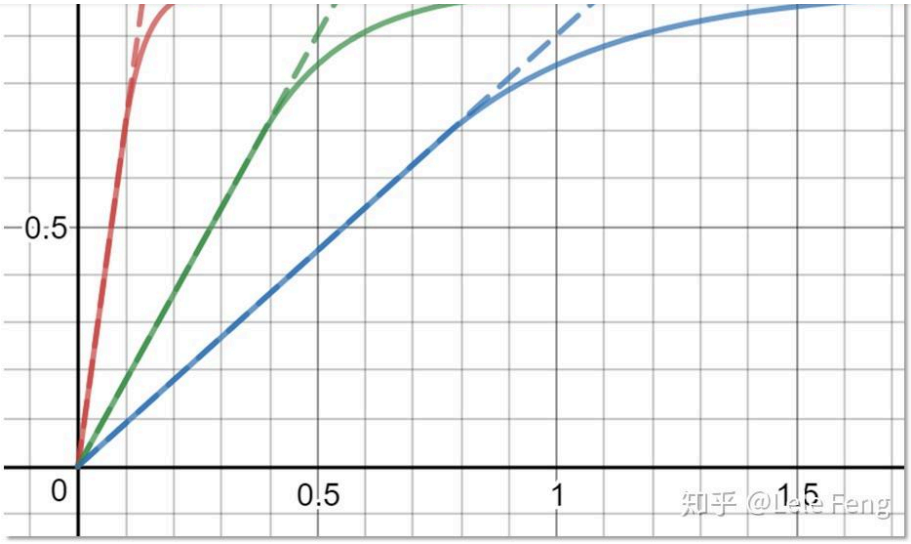
完成HDR Range Reduction之后，最后一步直接使用PQ OETF来进行最后的输出信号编码。

SDR显示下的Display Mapping是另一套逻辑。它的两个部分分别为：

- 使用一条固定的曲线计算SDR Range Reduction
- 使用颜色变换矩阵将颜色空间从BT. 2020转换到BT. 709，再使用BT.1886 OETF将结果最终转换到输出设备的颜色空间下

作者使用了线性函数段部分 + 指数shoulder的falloff部分来作为这条映射曲线：

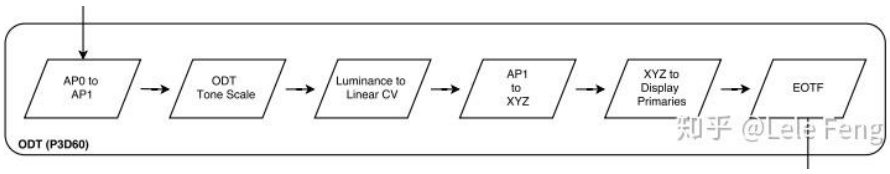
知乎 关注 推荐 热榜 专栏 圈子New 付费咨询 知学堂 直答



来源：HDR in Call of Duty

由于对RGB通道分别应用不同的曲线映射导致色调发生偏移，为了解决这个问题作者会单独对纯亮度曲线进行同样的曲线映射得到一个色调无偏的结果，然后在两个结果之间做插值。SDR也会使用之前类似的用户校准参数，但只用于调整黑点。最后，应用BT. 1886 OETF将进行输出信号的编码。

COD: WW II的这个Display Mapping阶段很像原生的ACES ODT阶段。ACES ODT也有相当于Range Reduction和应用EOTF编码信号的阶段。为了完整性这里给出ACES ODT（以P3D60为例）完整的工作流如下所示：



上图中的ODT tonescale就相当于COD这里的Range Reduction工作，但它的实现和COD不一样。这条ODT tonescale曲线同样是由分段的B-spine曲线定义而得（详见ACESlib.Tonescales.ctl文件中的segmented_spline_c9_fwd函数）。

AA和UI

Temporal AA[+]是在display mapping之后完成的，这是为了让AA可以在视觉线性空间下进行计算而避免额外的颜色空间转换和参数调整。

最后一步就是渲染UI了。UI的渲染方式有两种：

- 一种是先把UI渲染到offscreen buffer里，然后合成到最终图像上
- 一种是直接把UI渲染到backbuffer里

COD: WW II选择了第二种方式。这里也可以分为SDR和HDR两条逻辑线：

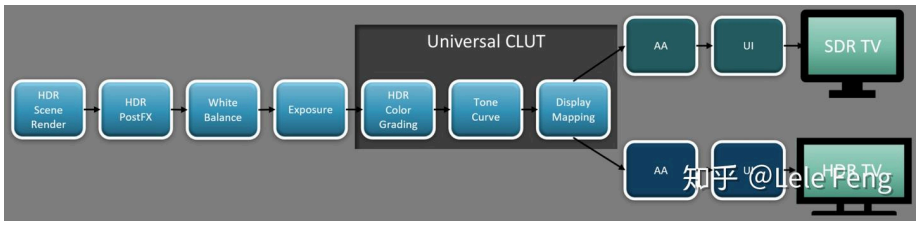
- 在SDR输出路径中，逻辑和老的管线保持一致，即直接混合到sRGB颜色空间下
- 在HDR输出路径中，会先把UI的亮度值缩放到最大300尼特的范围，然后应用颜色空间转换将其转换到BT. 2020颜色空间下，再应用PQ曲线映射

知乎 关注 推荐 热榜 专栏 圈子 New 付费咨询 知学堂 直答

幸运地是WW II的UI只需要常规的半透混合模式即可。

CLUT

在实现上，COD使用了一张被他们成为Universal CLUT+的LUT来完成上述管线的颜色转换操作，这可以大大提升整个管线的性能：



来源：HDR in Call of Duty

这张CLUT（Color LUT）的输入是经过曝光参数缩放过的线性场景颜色值，输出是output-referred的颜色值。这张CLUT包含了以下颜色操作：

- HDR Color Grading
- HDR Tone Curve（RRT）
- Display Mapping（ODT）

CLUT本质上就是一张3D纹理，它是在每一帧绘制的早期阶段靠一个async compute shader计算而得。作者使用log2函数来把输入的曝光后的HDR颜色值转换到纹理坐标空间的0~1范围内。据此来在这张3D纹理中查找其转换后的颜色值。

HDR in UE4

Talk is cheap, show me the code. 虽然我们没有COD的源码，但我们有UE4啊！接下来，我们会结合之前了解的所有理论知识，看看UE4里是如何实现HDR显示的。

总体来说，UE4的HDR颜色管线和WW II的逻辑大体一致，但这一些细节上没有WW II支持地那么好。我们一步步来看。

如何使用

在UE4里使用HDR显示还是非常简单的。在UE4文档中有比较详细的说明，总体来说使用三个命令行参数即可：

- r.HDR.EnableHDROutput：控制是否开启HDR输出，主要为了控制DXGI设置来改变backbuffer的格式，以便可以把HDR格式的backbuffer发送给HDR显示设备
- r.HDR.Display.OutputDevice：控制使用哪种输出设备的传递函数。UE4目前支持
- r.HDR.Display.ColorGamut：控制使用哪种色彩空间作为输出设备的颜色空间

外加两个和UI渲染相关的调整参数：

- r.HDR.UI.CompositeMode：控制是否开启UI的HDR合成模式，来尝试得到和SDR一致的UI视觉效果
- r.HDR.UI.Level：控制UI合成的高亮值

除了r.HDR.EnableHDROutput控制台参数，其他参数都是控制UE4渲染逻辑的shader参数。在我们之前的开发过程中，r.HDR.EnableHDROutput很容易引起程序崩溃，或导致HDR无法激活成功。正常情况下，当我们把r.HDR.EnableHDROutput设成1后，UE4会尝试在当前的平台下开启HDR，以D3D1

知乎 关注 推荐 热榜 专栏 圈子 New 付费咨询 知学堂 直答

```
    const float DisplayMaxOutputNits = (OutputDevice == DF_ACES2000_ST_2084 || OutputDevice == DF_ACES2000_ScRGB) ? 2000.f : 1000.f;
    const float DisplayMinOutputNits = 0.0f;    // Min output of the display
    const float DisplayMaxCLL = 0.0f;           // Max content light level in lumens (0.0 == unknown)
    const float DisplayFALL = 0.0f;             // Frame average light level (0.0 == unknown)

    // Ideally we can avoid setting TV meta data and instead the engine can do tone mapping based on the
    // actual current display properties (display mapping).
    static const auto CVarHDRColorGamut = IConsoleManager::Get().FindTConsoleVariableDataInt(TEXT("r.HDR.Display.ColorGamut"));
    const EDisplayGamut DisplayGamut = EDisplayGamut(CVarHDRColorGamut->GetValueOnAnyThread());
    SetHDRTVMode(true,
        DisplayGamut,
        DisplayMaxOutputNits,
        DisplayMinOutputNits,
        DisplayMaxCLL,
        DisplayFALL);

    // Ensure we have the correct color space set.
    EnsureColorSpace(DisplayGamut, OutputDevice);
}
}
```

知乎 @Lele Feng

上面的SetHDRTVMode和EnsureColorSpace都会根据当前的命令行参数来设置相应的DXGI参数。SetHDRTVMode函数负责根据当前的r.HDR.Display.OutputDevice和r.HDR.Display.ColorGamut参数为HDR meta数据设置相应的颜色空间三原色、白点值和输出的最大最小亮度值，以便把相应的HDR meta数据发送给当前的HDR显示设备：

```
void FD3D12Viewport::SetHDRTVMode(bool bEnableHDR, EDisplayGamut DisplayGamut, float MaxOutputNits, float MinOutputNits, float MaxCLL, float MaxFALL)
{
    ensure(SwapChain4.GetReference());

    static const DisplayChromacities DisplayChromacityList[] =
    {
        { 0.64000f, 0.33000f, 0.30000f, 0.60000f, 0.15000f, 0.06000f, 0.31270f, 0.32900f }, // DG_Rec709
        { 0.68000f, 0.32000f, 0.26500f, 0.69000f, 0.15000f, 0.06000f, 0.31270f, 0.32900f }, // DG_DCI-P3 D65
        { 0.70800f, 0.29200f, 0.17000f, 0.79700f, 0.13100f, 0.04600f, 0.31270f, 0.32900f }, // DG_Rec2020
        { 0.73470f, 0.26530f, 0.00000f, 1.00000f, 0.00010f,-0.07700f, 0.32168f, 0.33767f }, // DG_ACES
        { 0.71300f, 0.29300f, 0.16500f, 0.83000f, 0.12800f, 0.04400f, 0.32168f, 0.33767f }, // DG_ACEScg
    };

    if (bEnableHDR)
    {
        const DisplayChromacities& Chroma = DisplayChromacityList[DisplayGamut];

        // Set HDR meta data
        DXGI_HDR_METADATA_HDR10 HDR10MetaData = {};
        HDR10MetaData.RedPrimary[0] = static_cast<uint16>(Chroma.RedX * 50000.0f);
        HDR10MetaData.RedPrimary[1] = static_cast<uint16>(Chroma.RedY * 50000.0f);
        HDR10MetaData.GreenPrimary[0] = static_cast<uint16>(Chroma.GreenX * 50000.0f);
        HDR10MetaData.GreenPrimary[1] = static_cast<uint16>(Chroma.GreenY * 50000.0f);
        HDR10MetaData.BluePrimary[0] = static_cast<uint16>(Chroma.BlueX * 50000.0f);
        HDR10MetaData.BluePrimary[1] = static_cast<uint16>(Chroma.BlueY * 50000.0f);
        HDR10MetaData.WhitePoint[0] = static_cast<uint16>(Chroma.WpX * 50000.0f);
        HDR10MetaData.WhitePoint[1] = static_cast<uint16>(Chroma.WpY * 50000.0f);
        HDR10MetaData.MaxMasteringLuminance = static_cast<uint32>(MaxOutputNits * 10000.0f);
        HDR10MetaData.MinMasteringLuminance = static_cast<uint32>(MinOutputNits * 10000.0f);
        HDR10MetaData.MaxContentLightLevel = static_cast<uint16>(MaxCLL);
        HDR10MetaData.MaxFrameAverageLightLevel = static_cast<uint16>(MaxFALL);

        VERIFYD3D12RESULT(SwapChain4->SetHDRMetaData(DXGI_HDR_METADATA_TYPE_HDR10, sizeof(HDR10MetaData), &HDR10MetaData));
        UE_LOG(LogD3D12RHI, Log, TEXT("Setting HDR meta data on swap chain (%#016llx) using DisplayGamut %u:"), SwapChain4.GetReference(), static_cast<uint32>(DisplayGamut));
        UE_LOG(LogD3D12RHI, Log, TEXT("\t\tMaxMasteringLuminance = %.4f nits"), HDR10MetaData.MaxMasteringLuminance * .0001f);
        UE_LOG(LogD3D12RHI, Log, TEXT("\t\tMinMasteringLuminance = %.4f nits"), HDR10MetaData.MinMasteringLuminance * .0001f);
        UE_LOG(LogD3D12RHI, Log, TEXT("\t\tMaxContentLightLevel = %u nits"), HDR10MetaData.MaxContentLightLevel);
        UE_LOG(LogD3D12RHI, Log, TEXT("\t\tMaxFrameAverageLightLevel %u = nits"), HDR10MetaData.MaxFrameAverageLightLevel);
        bHDRMetaDataSet = true;
    }
    else
    {
        if (bHDRMetaDataSet)
        {
            // Clear meta data.
            VERIFYD3D12RESULT(SwapChain4->SetHDRMetaData(DXGI_HDR_METADATA_TYPE_NONE, 0, nullptr));
            UE_LOG(LogD3D12RHI, Log, TEXT("Clearing HDR meta data on swap chain (%#016llx)."), SwapChain4.GetReference());
            bHDRMetaDataSet = false;
        }
    }
}
```

知乎 @Lele Feng

EnsureColorSpace函数负责设置正确的backbuffer颜色空间格式：

```
void FD3D12Viewport::EnsureColorSpace(EDisplayGamut DisplayGamut, EDisplayFormat OutputDevice)
{
    ensure(SwapChain4.GetReference());

    DXGI_COLOR_SPACE_TYPE NewColorSpace = DXGI_COLOR_SPACE_RGB_FULL_G22_NONE_P709;  // sRGB;
    const bool bPrimaries2020 = (DisplayGamut == DG_Rec2020);

    // See console variable r.HDR.Display.OutputDevice.
    switch (OutputDevice)
    {
        // Gamma 2.2
    case DF_sRGB:
    case DF_Rec709:
        NewColorSpace = bPrimaries2020 ? DXGI_COLOR_SPACE_RGB_FULL_G22_NONE_P2020 : DXGI_COLOR_SPACE_RGB_FULL_G22_NONE_P709;
        break;

        // Gamma ST.2084
    case DF_ACES1000_ST_2084:
    case DF_ACES2000_ST_2084:
        NewColorSpace = DXGI_COLOR_SPACE_RGB_FULL_G2084_NONE_P2020;
        break;

        // Gamma 1.0 (Linear)
    case DF_ACES1000_ScRGB:
    case DF_ACES2000_ScRGB:
        // Linear. Still supports expanded color space with values >1.0f and <0.0f.
        // The actual range is determined by the pixel format (e.g. a UNORM format can only ever have 0-1).
        NewColorSpace = DXGI_COLOR_SPACE_RGB_FULL_G10_NONE_P709;
        break;
    }

    if (ColorSpace != NewColorSpace)
    {
        uint32 ColorSpaceSupport = 0;
        if (SUCCEEDED(SwapChain4->CheckColorSpaceSupport(NewColorSpace, &ColorSpaceSupport)) &&
            ((ColorSpaceSupport & DXGI_SWAP_CHAIN_COLOR_SPACE_SUPPORT_FLAG_PRESENT) != 0))
        {
            VERIFYD3D12RESULT(SwapChain4->SetColorSpace1(NewColorSpace));
            UE_LOG(LogD3D12RHI, Log, TEXT("Setting color space on swap chain (%#016llx): %s"), SwapChain4.GetReference(), *GetDXGIColorSpaceString(NewColorSpace));
            ColorSpace = NewColorSpace;
        }
    }
}
```

知乎 @Lele Feng

与COD: WW II可以让用户指定输出设备最小最大亮度值不同，UE4目前只支持固定的最大为1000或2000尼特的亮度值，这意味着UE4的ODT实现与WW II是不同的，UE4的逻辑会更加简单粗暴，后面会详细讲到。

知乎 关注 推荐 热榜 专栏 圈子 New 付费咨询 知学堂 直答

辑。UE4 目前共支持 7 种输出设备种类：

r.HDR.Display.OutputDevice	This is the device format of the output display
0	sRGB (LDR) (default)
1	Rec709 (LDR)
2	Explicit gamma mapping (LDR)
3	ACES 1000-nit ST-2084 (Dolby PQ) (HDR)
4	ACES 2000-nit ST-2084 (Dolby PQ) (HDR)
5	ACES 1000-nit ScRGB (HDR)
6	ACES 2000-nit ScRGB (HDR)

知乎 @Lele Feng

其中，r.HDR.Display.OutputDevice ≤ 2的部分对应了SDR输出设备，其余对应HDR输出设备，在代码里也会靠这个值的大小判断是走SDR逻辑还是HDR逻辑分支。

r.HDR.Display.ColorGamut控制输出设备的颜色空间，这会影响ODT阶段使用的颜色空间转换等逻辑。UE4目前支持5种显示颜色空间种类：

r.HDR.Display.ColorGamut	This is the color gamut of the output display.
0	Rec709 / sRGB, D65 (default)
1	DCI-P3, D65
2	Rec2020 / BT2020, D65
3	ACES, D60
4	ACEScg, D60

知乎 @Lele Feng

其中，r.HDR.Display.ColorGamut ≤ 1的部分对应了SDR颜色空间，其余对应HDR颜色空间。

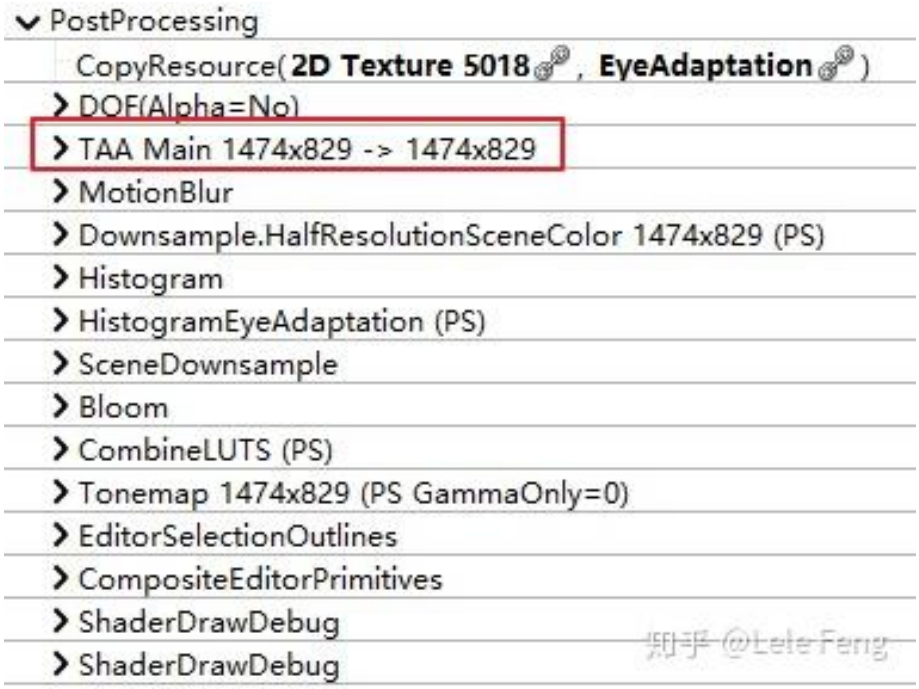
了解了这些基本参数后，现在我

知乎 关注 推荐 热榜 专栏 圈子 New 付费咨询 知学堂 直答

这部分逻辑比较常规，通常就是sRGB纹理输入进行基于物理的灯光渲染，得到场景亮度范围很大的HDR场景颜色值，亮度峰值可高达上万尼特，输出到下一阶段。这一部分所在的颜色空间的sRGB线性空间。

HDR Post Processing

UE4的后处理都是在HDR线性空间下计算的，包括AA。事实上，UE4的AA发生在后处理非常前面的阶段，仅位于DOF之后。以下是UE4.25在PC DX11下的Renderdoc截帧：



生成CLUT

首先是生成当前帧的CLUT，这部分逻辑在PostProcessCombineLUTs.usf里处理，有PS和CS两个版本，可以根据当前平台进行选择。这张CLUT包含了以下颜色操作：

- White Balance
- HDR Color Grading
- Output Transform（RRT + ODT）

纹理坐标转换到线性颜色值

代码首先将当前LUT的坐标转换到该位置对应的线性场景颜色：

```
float4 CombineLUTsCommon(float2 InUV, uint InLayerIndex)
{
    ...
    // construct the neutral color from a 3d position volume texture
    float4 Neutral;
    {
        float2 UV = InUV - float2(0.5f / LUTSize, 0.5f / LUTSize);

        Neutral = float4(UV * LUTSize / (LUTSize - 1), InLayerIndex / (LUT
    }
    ...
    float3 LUTEncodedCol
    float3 LinearColor;
```

知乎 关注 推荐 热榜 专栏 圈子New 付费咨询 知学堂 直答

```
        // Since ST2084 returns linear values in nits, divide by a scale fa
        // the reference nit result to be 1.0 in linear.
        // (for efficiency multiply by precomputed inverse)
        LinearColor = ST2084ToLinear(LUTEncodedColor) * LinearToNitsScaleIn
    }
    // Decode log values
    else
        LinearColor = LogToLin( LUTEncodedColor ) - LogToLin( 0 );
```

这里涉及两个分支：当OutputDevice（即r.HDR.Display.OutputDevice） >= 3，对应了HDR输出逻辑，这里就使用了在前一篇文章中提到的ST 2084 / PQ传递函数来进行从0~1范围重新映射到0~100尼特的转换。注意到这里使用了一个LinearToNitsScaleInverse值进行缩放，简单来说这是为了更加充分地利用PQ编码的LUT空间，因为PQ传递函数会把输入信号1映射到10000尼特，这基本大大超过了经过曝光缩放后的场景亮度值，也就是说，如果场景的最大亮度为100尼特左右，整个LUT的有效编码部分只占了不到50%，这是对LUT空间的一种浪费。UE4选择使用100作为一个缩放值，这样100尼特就对应了坐标为1的LUT像素部分：

```
// Scale factor for converting pixel values to nits.
// This value is required for PQ (ST2084) conversions, because PQ linear va
// The purpose is to make good use of PQ lut entries. A scale factor of 100
// about half of the PQ lut indexing below 1.0, with the other half for inp
// Also, 100nits is the expected monitor brightness for a 1.0 pixel value w
static const float LinearToNitsScale = 100.0;
static const float LinearToNitsScaleInverse = 1.0 / 100.0;
```

SDR输出设备分支则使用了log进行编码，使用log函数将0~1范围重新映射到0~场景最大像素值（大约为50）的转换：

```
float3 LogToLin( float3 LogColor )
{
    const float LinearRange = 14;
    const float LinearGrey = 0.18;
    const float ExposureGrey = 444;

    // Using stripped down, 'pure log', formula. Parameterized by grey poin
    float3 LinearColor = exp2( ( LogColor - ExposureGrey / 1023.0 ) * Linea
    //float3 LinearColor = 2 * ( pow(10.0, ((LogColor - 0.616596 - 0.03) /
    //float3 LinearColor = ( pow( 10, ( 1023 * LogColor - 685 ) / 300) - .0
    //LinearColor = max( 0, LinearColor );

    return LinearColor;
}
```

应用White Balance

在上述转换到的线性场景值下计算White Balance：

```
float4 CombineLUTsCommon(float2 InUV, uint InLayerIndex)
{
    ...
    float3 BalancedColor = WhiteBalance( LinearColor );
```

sRGB转换到AP1颜色空间

知乎 关注 推荐 热榜 专栏 圈子New 付费咨询 知学堂 直答

```
float4 CombineLUTsCommon(float2 InUV, uint InLayerIndex)
{
    ...
    float3 ColorAP1 = mul( sRGB_2_AP1, BalancedColor );
```

回顾在上一篇文章中的内容，AP1颜色空间的色域范围与Rec. 2020非常接近，它更适合进行CG和VFX计算，得到的结果更接近光谱渲染的ground truth结果。

尽管我们把色域从sRGB空间变换到了广色域AP1下，但由于之前的场景渲染都是在sRGB线性颜色空间下进行的，我们得到的色度值始终都是在sRGB色度范围内。这里UE4有个trick，它会假装场景是在一个Wide Color Space下计算的，其三原色介于P3和AP1之间，然后使用Wide_2_AP1进行颜色转换，最后使用参数与原先sRGB_2_AP1的转换结果进行插值：

```
float4 CombineLUTsCommon(float2 InUV, uint InLayerIndex)
{
    ...
    // Expand bright saturated colors outside the sRGB gamut to fake wide g
    if (!bUseMobileTonemapper)
    {
        float  LumaAP1 = dot( ColorAP1, AP1_RGB2Y );
        float3 ChromaAP1 = ColorAP1 / LumaAP1;

        float ChromaDistSqr = dot( ChromaAP1 - 1, ChromaAP1 - 1 );
        float ExpandAmount = ( 1 - exp2( -4 * ChromaDistSqr ) ) * ( 1 - exp

        // Bizarre matrix but this expands sRGB to between P3 and AP1
        // CIE 1931 chromaticities: x       y
        //              Red:        0.6965  0.3065
        //              Green:      0.245   0.718
        //              Blue:       0.1302  0.0456
        //              White:      0.3127  0.329
        const float3x3 Wide_2_XYZ_MAT =
        {
            0.5441691,  0.2395926,  0.1666943,
            0.2394656,  0.7021530,  0.0583814,
            -0.0023439,  0.0361834,  1.0552183,
        };

        const float3x3 Wide_2_AP1 = mul( XYZ_2_AP1_MAT, Wide_2_XYZ_MAT );
        const float3x3 ExpandMat = mul( Wide_2_AP1, AP1_2_sRGB );

        float3 ColorExpand = mul( ExpandMat, ColorAP1 );
        ColorAP1 = lerp( ColorAP1, ColorExpand, ExpandAmount );
    }
```

插值使用的参数是在后处理组件中的Expand Gamut值控制的，默认值为1：

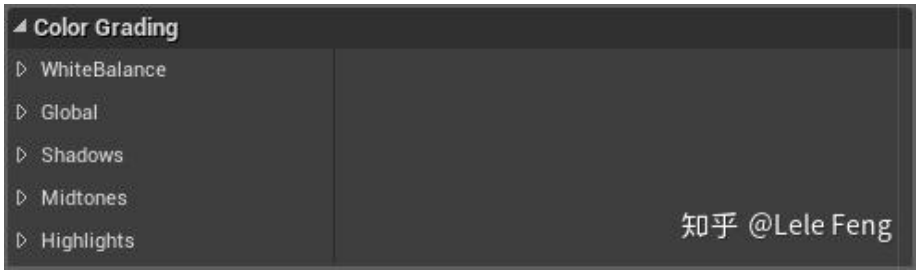
知乎 关注 推荐 热榜 专栏 圈子New 付费咨询 知学堂 直答

应用Color Grading

接着，在AP1空间下计算Color Grading：

```
float4 CombineLUTsCommon(float2 InUV, uint InLayerIndex)
{
    ...
    ColorAP1 = ColorCorrectAll( ColorAP1 );
```

UE4的Color Grading参数还是比较丰富的，通常来说，我们建议任何和风格化调色相关的调整都应该靠Color Grading去完成。Color Grading或Color Correction提供了一种让美术逐镜头逐场景调整光照色调的途径。UE4在Post Process Volume里提供了大量类似后期调色功能的参数：



至此，所有与美术风格相关的计算和调整都已完成，接下来就是根据输出设备计算Output Transform，这部分大量使用了ACES的实现逻辑。这里可以分为SDR和HDR两条逻辑路径。

SDR下的Output Transform

SDR下的Output Transform使用了ACES的LMT + RRT + ODT变换。首先是LMT部分，UE4使用了ACES LMT中的Blue Light Artifact Fix部分来修正高亮度的蓝色值导致的过饱和问题：

```
float4 CombineLUTsCommon(float2 InUV, uint InLayerIndex)
{
    ...
    const float3x3 BlueCorrect =
    {
        0.9404372683, -0.0183068787, 0.0778696104,
        0.0083786969,  0.8286599939, 0.1629613092,
        0.0005471261, -0
    };
```

知乎 关注 推荐 热榜 专栏 圈子New 付费咨询 知学堂 直答

```
        -0.0106337,   1.20632,   -0.19569,
        -0.000590887, 0.00105248, 0.999538
    };
    const float3x3 BlueCorrectAP1    = mul( AP0_2_AP1, mul( BlueCorrect,
    const float3x3 BlueCorrectInvAP1 = mul( AP0_2_AP1, mul( BlueCorrectInv

    // Blue correction
    ColorAP1 = lerp( ColorAP1, mul( BlueCorrectAP1, ColorAP1 ), BlueCorrec
```

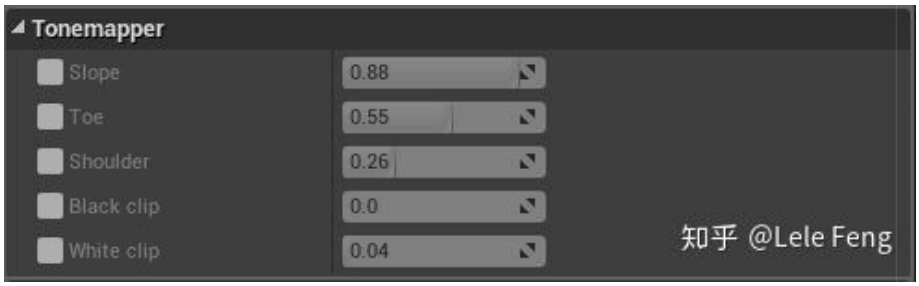
这部分没啥特别要解释的，就是“标准”。矫正程度参数可以在后处理组件中的Blue Correction值控制的，默认值为0.6。

接下来是计算Tonemapping：

```
float4 CombineLUTsCommon(float2 InUV, uint InLayerIndex)
{
    ...
    // Tonemapped color in the AP1 gamut
    ColorAP1 = FilmToneMap( ColorAP1 );
```

需要注意的是，Tonemapping的具体含义在不同引擎和技术分享中有些许差别，虽然没什么本质上的差别，但为了防止迷惑我们还是多说两句。在COD: WW II的分享中，Tonemapping统称包含了Tone Curve（RRT）和Display Mapping（ODT）的操作。而在UE4里，Tonemapping指的是RRT和ODT合并之后统一进行的曲线映射部分，即直接在Output Transform阶段使用一条S曲线进行重新映射，不再严格地单独区分RRT和ODT阶段。UE4的Tonemapping逻辑都在FilmToneMap函数中处理，这部分代码比较长就不粘贴了，总体来说就是结合了ACES RRT阶段和ACES SDR ODT阶段的实现逻辑。

UE4允许我们在Post Process Volume里的Tonemapper里调整这条S曲线参数，这些参数的默认值与ACES的实现保持一致，因此如果没有特殊需求，我们不应该逐场景逐镜头地调整下列参数，而是使用Color Grading参数来进行所有的艺术风格化控制：



最后是应用OETF传递函数进行编码。在这之前，为了保证白点的准确性先抵消之前LMT所作的Blue Correction操作，然后把颜色空间重新变换回sRGB颜色空间为最后输出做准备：

```
float4 CombineLUTsCommon(float2 InUV, uint InLayerIndex)
{
    ...
    // Uncorrect blue to maintain white point
    ColorAP1 = lerp( ColorAP1, mul( BlueCorrectInvAP1, ColorAP1 ), BlueCor

    // Convert from AP1 to sRGB and clip out-of-gamut values
    float3 FilmColor = max(0, mul( AP1_2_sRGB, ColorAP1 ));
```

为了方便解释，我们把SDR输出设备相关的OETF计算放到一起来看，总体而言就是根据不同输出设备选择使用不同的OETF传递函数：

知乎 关注 推荐 热榜 专栏 圈子New 付费咨询 知学堂 直答

```
    ...
    half3 OutDeviceColor = 0;
    // sRGB, user specified gamut
    if( GetOutputDevice() == 0 )
    {
        // Convert from sRGB to specified output gamut
        float3 OutputGamutColor = mul( AP1_2_Output, mul( sRGB_2_AP1, Film

        // Apply conversion to sRGB (this must be an exact sRGB conversion
        OutDeviceColor = LinearToSrgb( OutputGamutColor );
    }
    // Rec 709, user specified gamut
    else if( GetOutputDevice() == 1 )
    {
        // Convert from sRGB to specified output gamut
        float3 OutputGamutColor = mul( AP1_2_Output, mul( sRGB_2_AP1, Film

        // Didn't profile yet if the branching version would be faster (di
        OutDeviceColor = LinearTo709Branchless( OutputGamutColor );
    }
    // OutputDevice == 2
    // Gamma 2.2, user specified gamut
    else if ( GetOutputDevice() == 2 )
    {
        // Convert from sRGB to specified output gamut
        float3 OutputGamutColor = mul( AP1_2_Output, mul( sRGB_2_AP1, Film

        // This is different than the prior "gamma" curve adjustment (but
        // For displays set to a gamma colorspace.
        // Note, MacOSX native output is raw gamma 2.2 not sRGB!
        OutDeviceColor = pow( OutputGamutColor, InverseGamma.z );
    }

    // Better to saturate(lerp(a,b,t)) than lerp(saturate(a),saturate(b),t
    OutColor.rgb = OutDeviceColor / 1.05;
    OutColor.a = 0;

    return OutColor;
}
```

经过不同的OETF编码后的信息已经准备好可以发送给对应的SDR显示设备了。

HDR下的Output Transform

相比于SDR Output Transform，UE4的HDR Output Transform少了很多变换。UE4的HDR输出管线下没有实现类似Tonemapping的计算，而是在算完Color Grading后，直接把颜色空间从AP1重新变换到sRGB下：

```
float4 CombineLUTsCommon(float2 InUV, uint InLayerIndex)
{
    ...
    ColorAP1 = ColorCorrectAll( ColorAP1 );

    // Store for Legacy tonemap later and for Linear HDR output without to
    float3 GradedColor = mul( AP1_2_sRGB, ColorAP1 );
```

它的Output Transform基本就是直接进行ODT变换，其本质同样是使用不同的OETF函数进行输出信号的编码：

知乎 关注 推荐 热榜 专栏 圈子New 付费咨询 知学堂 直答

```
    ...
    if( GetOutputDevice() == 3 || GetOutputDevice() == 5 )
    {
        // 1000 nit ODT
        float3 ODTColor = ACESOutputTransforms1000( GradedColor );

        // Convert from AP1 to specified output gamut
        ODTColor = mul( AP1_2_Output, ODTColor );

        // Apply conversion to ST-2084 (Dolby PQ)
        OutDeviceColor = LinearToST2084( ODTColor );
    }

    // ACES 2000nit transform with PQ/2084 encoding, user specified gamut
    else if( GetOutputDevice() == 4 || GetOutputDevice() == 6 )
    {
        // 2000 nit ODT
        float3 ODTColor = ACESOutputTransforms2000( GradedColor );

        // Convert from AP1 to specified output gamut
        ODTColor = mul( AP1_2_Output, ODTColor );

        // Apply conversion to ST-2084 (Dolby PQ)
        OutDeviceColor = LinearToST2084( ODTColor );
    }

    else if( GetOutputDevice() == 7 )
    {
            float3 OutputGamutColor = mul( AP1_2_Output, mul( sRGB_2_AP1, (
            OutDeviceColor = LinearToST2084( OutputGamutColor );
    }

    // Better to saturate(lerp(a,b,t)) than lerp(saturate(a),saturate(b),t
    OutColor.rgb = OutDeviceColor / 1.05;
    OutColor.a = 0;

    return OutColor;
}
```

最后，使用PQ的OETF传递函数对信号进行编码，以便把编码后的HDR信号传递给HDR显示设备。

使用CLUT

使用CLUT的逻辑是在PostProcessTonemap.usf里处理，其主要函数逻辑在TonemapCommonPS函数中。TonemapCommonPS函数首先会计算之前还未完成的后处理，包括Grain、Color Fringe、Sharpen、Bloom、Exposure、Vignette等，将上述后处理计算得到的最终结果转换为3D LUT的采样坐标进行颜色查找：

```
float4 TonemapCommonPS()
{
    // Compute Grain/Color Fringe/Sharpen/Bloom/Exposure/Vignette
    ...
    half3 OutDeviceColor = ColorLookupTable( LinearColor );
    ...
    return OutColor;
}
```

知乎 关注 推荐 热榜 专栏 圈子New 付费咨询 知学堂 直答

UE4里的ACES

除去UE4对Blue Light Artifact Fix的LMT部分和Fake Wide Gamut的trick部分，UE4在SDR下的实现与ACES sRGB几乎一模一样。这给了我们在DCC软件里校准UE4渲染效果一条可选路径，如果不想实现完全自定义OCIO文件或其他LUT文件，我们可以关闭UE4的这两项修改（在后处理组件中把Blue Correction和Expand Gamut的值设成0），再使用ACES RRT + sRGB ODT相关的OCIO即可。

UE4和COD的HDR实现对比

可以发现，UE4和COD在HDR显示管线上的实现还是有些不同的地方，具体表现在：

- WW II的Output Transform比UE4的处理要更加复杂：
  - UE4总体来说就是直接使用了ACES的原生代码逻辑，在SDR路径下糅合了ACES RRT和ODT的Tonemapping计算将其嵌套在了UE4的渲染管线里。但在HDR路径下只保留了ACES最基本的OETF部分，且只支持固定的1000尼特和2000尼特两种最大亮度值，这意味着开启HDR后我们可能需要手动进行HDR Tonemapping调整才能尽可能得到和SDR一致的画面效果
  - WW II针对他们的需求做了更多的适应性工作，他们仍然保留了分明的RRT和ODT阶段。在RRT阶段使用了一条与ACES RRT非常相似的Tone Curve进行调整，让经过映射后的场景亮度处于PQ的0~10000尼特范围内。在ODT阶段同样保留了类似ACES ODT的Display Mapping计算，即在应用设备的OETF之前，WW II在SDR和HDR下各自使用了一条相互独立的S曲线来做专门的HDR Range Reducion，这个过程可以让用户参与调整参数，来处理不同显示设备的显示参数（最小和最大亮度值等），具有更大的可控性
- WW II的UI绘制是直接混合到backbuffer里，而UE4会先输出到offscreen buffer里再混合到backbuffer里

总体来说，UE4目前的实现给了我们一个基底，要想实现效果更好的HDR显示还需要进行一定的二次开发，例如使用UE4开发的战争机器5在HDR显示上相比于UE4的原生实现明显好了不少。战争机器5在采访中曾透露他们使用的tonemapping是使用了微软第一方游戏中大量的HDR/SDR图像作为训练样本、利用机器学习来训练得到的，在此基础上还支持玩家针对他们当前的显示设备进行游戏内的校准。

写在最后

尽管在提到HDR显示的时候总是会提到ACES，但从这段时间的学习也逐渐意识到ACES并不是游戏HDR显示里所谓的银弹。在工作中也遇到过如何选择Tonemapping的时候，Angelo Pesce在他的博客里有曾发出这样的疑问。ACES Tonemapping真的适合所有的游戏吗？尽管ACES的S曲线的各个参数有大量影视行业的大佬背书，这是他们经过大量图像的验证过后得到的“最合理”的参数，但这种“高级的影视感”可能并不适合所有的游戏。除了视觉上的“电影感”，ACES另一大优势就是标准化，但这一点在游戏行业是否有那么重要似乎也是一个值得争论的问题。一些游戏类型，例如二次元风格的游戏，它们涉及到的开发工具种类有限，使用一个和影视行业相同的标准似乎也完全没有必要。但不可否认，“标准化”会给开发人员更多的安全感，也是从小作坊走向工业化的重要标志，是适应ACES标准还是开发自己的标准，这是个问题。

虽然目前Unity和UE4都内置了完整的ACES，但环顾3A游戏在支持HDR显示上的分享，可以发现他们都在ACES的思想上或有多少有自己的修改，例如战争机器5使用的ML Tonemapping、WW II使用的基于BT2390 EETF标准的HDR Display Mapping。可以说，这些商业引擎里的实现只是给了我们一个起跑线，对于PBR类型的游戏来说，ACES可以和其他DCC软件具有更好的兼容性，ACES的画面电影感和一统江湖的标准化无疑是一个不错的开头。但对于目前国内的开发环境来说，HDR显示并没有被提升到一个重要的位置，同时完整的ACES实现对手机平台的计算

知乎 关注 推荐 热榜 专栏 圈子 New 付费咨询 知学堂 直答

虽然这是计划中的最后一篇，但因为是一边学习一边写博客的过程所以在编写过程中难免有些疏漏，也有一些想讲但不合适一股脑放在文章中的内容，可能之后会再写一篇查漏补缺，或者随时修改下之前文章。

米娜！感谢大家的阅读，希望大家开发顺利，身体健康！完结撒花！

参考文献

1. Digital Dragons 2018: HDR in Call of Duty

2. SIGGRAPH Asia 2018：Practical HDR and Wide Color Techniques in Gran Turismo SPORT

3. GDC 2019: Not-So-Little Light: Bringing 'Destiny 2' to HDR Displays

4. GDC 2018: Advances in the HDR Ecosystem

5. GDC 2017: HDR Dynamic Range Color Grading and Display in Frostbite

6. resetera.com/threads/hd...

7. docs.unrealengine.com/e...

8. developer.nvidia.com/hd...

所属专栏 · 2020-08-30 17:55 更新

RGBA
Lele Feng
21 篇内容 · 3929 赞同

订阅

最热内容 · [总结] 漫谈HDR和色彩管理（一）光、颜色和色度图

编辑于 2020-06-27 23:41

HDR RGB 色彩空间

多语种+7 大方言，ima让跨地域沟通无界限

ima打破语言和地域沟通壁垒，不仅支持中文、英文、粤语三大常用语种，还兼容四川、陕西、河南等 7 大方言，同时叠加实用的发言人识别功能。哪怕是跨地区团队沟通或多语种访谈，都能清晰捕捉每段发言，让信息传递零障碍，沟通协作更顺畅。 查看详情

ima 的广告

理性发言，友善互动

41 条评论 默认 最新

CGBull

知乎 关注 推荐 热榜 专栏 圈子 New 付费咨询 知学堂 直答

2020-06-27 回复 5

Lele Feng 作者

小谷子果然厉害直接发图片！

2020-06-27 回复 1

展开其他 3 条回复 〉

张晓宇

看完了整个系列，赞！

个人总结一下：老做法是把场景的luminance tone map到0-1，然后做oetf/gamma编码得到“感知上均匀的亮度”即lightness，通过8bit per channel的swapchain传给SDR显示器，最后eotf解码到250nits范围内，这时仍能保持差不多的uniform perception

ACES这套是把场景luminance先tone map到特定HDR显示器的范围，再做oetf/PQ编码得到lightness，通过10/12bits per channel的swapchain传给同一个显示器做eotf显示，从而在10000nits以内都能具有均匀的亮度视觉。

OETF/EOTF的本质是对线性物理量的编码或说量化：用最小的量化误差来达到“视觉亮度的均匀”，即量化要满足1%相对亮度规律(Weber's Law)。所以gamma,HLG和PQ这些OETF的目标是一致的，都是给定亮度范围，位深，和1.01阈值等要求，拟合得到的量化函数。个人感觉从这个角度讲可能更容易理解一些。

反而是tonemapping这步，感觉可能对color grading有不好的影响。校色放在tonemapping之前做吧，LUT范围太大了可能不精确，放在mapping之后做，LUT又会跟具体显示器相关了？

2022-02-10 回复 3

Mr.DJ

大佬 大佬

2025-04-30 回复 喜欢

Ainscel

完整阅读完了。
不过有个具体的疑惑，现在的游戏好像多数都不调用系统中的icc文件？那么他输出画面时直接按BT2020输出的话会导致欠饱和吧？
文中提到了使命召唤ww2的hdr实现中包括了转换到对应输出设备色彩空间的一环，它是从显示器edid内读取色域数据的吗？

2023-10-07 回复 喜欢

林书溪

正好这几天在看ue后处理的tonemap部分，自己看不是完全看得懂，女神的文章如醍醐灌顶

2022-11-29 回复 喜欢

晚晚

想请教下纹理坐标转换到线性颜色值不是输入的部分吗，为什么要通过判断output device来设置呢~ 谢谢楼主😘

2022-09-26 回复 喜欢

陌曦

女神

2022-08-01

知乎 关注 推荐 热榜 专栏 圈子 New 付费咨询 知学堂 直答

[redacted]

2021-08-26 回复 喜欢

风声蝉鸣

大佬问个问题，我在UE里新建一个饱和度为50%，亮度为1的材质球，不管怎么打灯，都无法实现亮度为1的时候饱和度为50%，并且随着灯光强度不一样，饱和度变化非常大。但是我使用贴图就没有这个问题，也是因为Tonemap的问题吗？

2021-05-13 回复 喜欢

qingdong

乐乐女神

2020-12-11 回复 喜欢

点击查看全部评论 ›

理性发言，友善互动

推荐阅读

如何在 HDRP 中实现光线追踪?

Unity... 发表于Unity...

漫谈HDR和色彩管理

张鑫 发表于UWA：简...

[学习笔记]HDR in Call of Duty(使命召唤中的高动态范围)

PZZZB 发表于PZB的图...

UE4 HDR LightMap压缩优化

ShaVenZz